

Projeto Zelo: Especificação Técnica e de Produto Completa

Este documento consolida de forma abrangente o planejamento estratégico, a especificação de produto, a engenharia de requisitos ágeis, a arquitetura de software e os padrões de governança/escrita para o desenvolvimento do **Projeto Zelo** (Sistema de Gerenciamento e Compartilhamento de Acervos). Esta estrutura está alinhada às exigências acadêmicas da UNIFAN (Trabalhos Discentes Efetivos de Métodos Ágeis e Banco de Dados Avançado).

1. Visão Geral do Produto e Estratégia (PRD)

O **Zelo** é um sistema web voltado para o gerenciamento de acervos pessoais ou compartilhados, permitindo cadastrar obras, controlar exemplares físicos, registrar empréstimos, acompanhar devoluções e gerenciar permissões de acesso.

1.1. O Problema Central

Colecionadores particulares e gestores de acervos bibliográficos residenciais que realizam empréstimos informais para amigos, familiares ou comunidades frequentemente perdem o controle sobre os seus itens. O problema real reside na falta de rastreabilidade e de ferramentas simples para catalogar volumes, estipular prazos de entrega e cobrar devoluções, resultando muitas vezes na perda definitiva de edições raras ou afetivas.

1.2. Proposta de Valor

"O Zelo ajuda pessoas a compartilharem os seus acervos sem perder o controle sobre as obras, exemplares, empréstimos e devoluções."

1.3. Escopo do MVP (Minimum Viable Product)

- Autenticação funcional de utilizadores (Login, Registo e Senhas protegidas).
- CRUD completo de Acervos (criação de bibliotecas lógicas/temáticas).
- CRUD de Obras e os seus respectivos Exemplares físicos.
- Fluxo de Registo de Empréstimo (vincular um exemplar a um tomador e definir prazo).
- Fluxo de Registo de Devolução (retorno do exemplar ao estado disponível).
- Painel visual básico para acompanhamento de prazos e pendências.

2. Engenharia de Requisitos e Backlog Ágil

Seguindo as premissas do *Product Lab* e frameworks ágeis (Scrum/Kanban), o backlog está estruturado em Épicos e Histórias de Utilizador (User Stories) com foco em valor contínuo.

2.1. Épico 1: Gestão de Acesso e Perfis

- **US01 - Registo de Utilizador:** Como um novo leitor ou colecionador, quero criar uma conta na plataforma informando nome, e-mail e senha, para que eu possa ter o meu ambiente exclusivo de acervos.
- **US02 - Login Seguro:** Como um utilizador registado, quero autenticar-me com e-mail e senha para aceder em segurança aos meus dados confidenciais e históricos.

2.2. Épico 2: Organização do Acervo

- **US03 - Criar Acervo:** Como um colecionador, quero criar um novo acervo (ex: "Livros de Ficção Científica", "Didáticos") definindo se ele é privado ou visível para convidados, para organizar os meus volumes por categorias.
- **US04 - Catalogar Obra e Exemplares:** Como administrador de um acervo, quero registar uma Obra (Título, Autor, ISBN, Ano) e indicar quantos Exemplares físicos possuo daquela mesma obra, para gerir a quantidade real disponível para partilha.

2.3. Épico 3: Operação de Empréstimos

- **US05 - Registrar Empréstimo:** Como dono do acervo, quero associar um exemplar físico a um amigo (tomador) com uma data limite de devolução, para que o sistema mude o estado do livro para "Emprestado".
- **US06 - Confirmar Devolução:** Como dono do acervo, quero dar baixa num empréstimo ativo quando o livro for devolvido, para que o exemplar volte a ficar disponível no catálogo.

3. Arquitetura de Software (Clean Architecture)

A organização do código do Projeto Zelo adota os princípios da **Clean Architecture (Arquitetura Limpa)**, adaptada para o ecossistema Python (Flask) e NoSQL (MongoDB). As dependências apontam estritamente de fora para dentro, isolando as regras de negócio de detalhes tecnológicos e de persistência de dados.

Camada	Responsabilidade Principal	Componentes Internos
1. Domínio (Entities)	Contém o núcleo do negócio. Modelos e validações puras, sem dependência de bibliotecas externas ou frameworks.	Usuario, Acervo, Obra, Empréstimo
2. Casos de Uso (Use	Orquestra as regras da	CriarAcervo,

Camada	Responsabilidade Principal	Componentes Internos
Cases)	aplicação. Executa o fluxo lógico necessário para completar uma ação de negócio.	ConcederEmprestimo, RegistrarDevolucao
3. Adaptadores (Adapters)	Converte os dados do formato mais conveniente para os Casos de Uso para o formato externo (e vice-versa).	Controllers da API REST, Definição de Interfaces de Repositórios (Gateways).
4. Infraestrutura (Frameworks & Drivers)	A camada mais externa. Onde os frameworks residem, rotas HTTP e acessos diretos ao banco de dados acontecem.	Rotas Flask, Conexão com o MongoDB (PyMongo), Drivers de Envio de E-mail.

4. Modelagem de Dados NoSQL (MongoDB)

Para cumprir o requisito de gestão de dados flexíveis e dinâmicos (orientado a documentos), optou-se pela estratégia de **Aninhamento/Embutimento (Embedding)** em relações de alta coesão, otimizando as leituras e eliminando a necessidade de JOINS complexos.

4.1. Estratégia de Embedding para Membros e Permissões

A relação entre Usuário e Acervo possui um atributo chamado "Papel" (Administrador ou Leitor). No MongoDB, isto materializa-se como um array de subdocumentos dentro da coleção de acervos, garantindo segurança e consulta instantânea.

4.2. Estrutura de Coleções Sugerida (JSON)

Coleção: usuarios

```
{
  "_id": ObjectId("60a7b1..."),
  "nome": "Mariana Souza",
  "email": "mariana@email.com",
  "senha_hash": "$2b$12$..."
}
```

Coleção: acervos

```
{
```

```

    "_id": ObjectId("60a7c5..."),
    "nome": "Biblioteca Pessoal Mariana",
    "visibilidade": "privado",
    "membros": [
      {
        "usuario_id": ObjectId("60a7b1..."),
        "papel": "Administrador"
      }
    ],
    "obras": [
      {
        "obra_id": ObjectId("de301..."),
        "titulo": "Dom Casmurro",
        "autor": "Machado de Assis",
        "exemplares": [
          { "codigo_exemplar": "DOM-01", "status": "disponivel" },
          { "codigo_exemplar": "DOM-02", "status": "emprestado" }
        ]
      }
    ]
  }
}

```

Coleção: empréstimos (Separada para evitar o crescimento indefinido e permitir auditoria de histórico geral)

```

{
  "_id": ObjectId("60a7d9..."),
  "acervo_id": ObjectId("60a7c5..."),
  "exemplar_codigo": "DOM-02",
  "tomador_nome": "Carlos Silva",
  "data_emprestimo": ISODate("2026-05-15T00:00:00Z"),
  "data_limite_devolucao": ISODate("2026-05-30T00:00:00Z"),
  "status": "ativo"
}

```

5. Padrões de Nomenclatura e Convenções de Escrita (Style Guide)

Para manter a consistência estética do repositório Git e a uniformidade de escrita entre todos os programadores do grupo, definem-se os seguintes padrões obrigatórios:

5.1. Padrão de Pastas e Diretórios (kebab-case)

Todas as pastas, diretórios estruturais e pacotes do projeto devem ser escritos estritamente em letras minúsculas separadas por hífen.

- *Exemplos:* use-cases/, data-sources/, web-adapters/

5.2. Padrão de Arquivos e Módulos Python (snake_case)

Todos os arquivos com a extensão .py e scripts de configuração gerais devem utilizar letras minúsculas separadas por sublinhado (_), em conformidade com as diretrizes da PEP 8.

- *Exemplos:* criar_acervo.py, usuario_repository.py, db_config.py

5.3. Classes, Entidades e Interfaces (PascalCase)

Todas as declarações de Classes estruturais, Entidades de domínio, Casos de Uso e Controllers devem iniciar com letra maiúscula, eliminando separadores. Cada palavra interna subsequente inicia também com maiúscula.

- *Exemplos:* class UsuarioLogado:, class ConcederEmprestimo:, class MongoObraRepository:

5.4. Variáveis, Funções, Métodos e Atributos (snake_case)

Os elementos dinâmicos e declarativos dentro do código fonte (variáveis de escopo local, parâmetros, nomes de funções, métodos de classe e atributos de instâncias) adotam minúsculas com sublinhado.

- *Exemplos:* data_devolucao = None, def validar_permissao_membro():

5.5. Constantes (UPPER_CASE_SNAKE_CASE)

Variáveis de configuração globais cujo valor permanece imutável por todo o ciclo de vida da aplicação devem ser escritas inteiramente em letras maiúsculas separadas por sublinhado.

- *Exemplos:* MONGO_URI = "mongodb://localhost:27017", PAPEL_ADMINISTRADOR = "administrador"

6. Governança e Qualidade (DoD e Critérios de Aceitação)

Para garantir que o produto seja entregue sem regressões e que atenda aos critérios rigorosos das bancas de avaliação da UNIFAN, aplicam-se as regras abaixo:

6.1. Critérios de Aceitação (Exemplo: Fluxo de Empréstimos)

- **Cenário 1:** O exemplar físico solicitado está com o estado "disponível". O sistema deve permitir o registo, criar o documento correspondente na coleção empréstimos e alterar o estado do exemplar para "emprestado".
- **Cenário 2:** O exemplar solicitado já possui um empréstimo ativo ("emprestado"). O sistema deve barrar a operação imediatamente e disparar um alerta visual para o

utilizador, impedindo a inconsistência de dados.

6.2. Definition of Done (DoD) - Definição de Concluído

Uma User Story ou tarefa técnica só poderá ser movida para a coluna "Done" (Concluído) se cumprir todos os seguintes pré-requisitos:

1. Código-fonte revisado e integrado via Pull Request no ramo principal (main/develop).
2. Nomenclatura validada de acordo com o Capítulo 5 deste guia (Style Guide).
3. Execução local ou em ambiente de testes validando os fluxos alternativos e de sucesso.
4. Persistência verificada diretamente na ferramenta visual do banco de dados (ex: MongoDB Compass) assegurando que os subdocumentos foram injetados no formato planeado.